

OutsAIdé



TEAM 18

Jenny Le | jenl

Laurynas Pilibaitis | laurynp

Daniel Duy Mai | daniedm

Abdirahman Abdulkadir Yusuf | abdirahy

Haakon Eide Hilmen | haakoehi

Alisa Sorano | alisamu

VEILEDERE

Viljar Hardersen | viljardh

Iuliia Perminova | iuliiape

Rapport i IN2000

Institutt for informatikk

16.05.2025

INNHOLDSFORTEGNELSE

1. Introduksjon	2
2. Innledning og prosjektbeskrivelse.....	3
2.1 Presentasjon av prosjektet.....	3
2.2 Kort beskrivelse av løsningen	3
2.3 Presentasjon av prosjektgruppen.....	3
3. Brukerdokumentasjon	4
3.1 Applikasjon og målgruppe	4
3.2 Applikasjonens funksjonalitet og brukerflyt	5
3.3 Tilgjengelige plattformer.....	6
3.4 Tilgang og bruk	7
4. Produktdokumentasjon	7
4.1 Applikasjonsstruktur	7
4.2 Kravspesifikasjon	8
4.3 Kvalitetssegenskaper.....	9
4.4 API-bruk og utfordringer.....	11
5. Prosessdokumentasjon	13
5.1 Utviklingsmetoder og smidige praksiser.....	13
5.2 Prosjektets gang	14
5.3 Designutvikling	16
5.4 Endringer i kravspesifikasjonen.....	17
5.5 Testing: planlegging, gjennomføring og verktøy	18
6. Refleksjon.....	19
6.1 Tverrfaglig samarbeid.....	19
6.2 Smidig arbeid og effekt	20
6.3 Utviklingsutfordringer og læring	21
7. Avslutning.....	22
8. Kilder og referanser.....	24
9. Vedlegg	25
9.1 Teamavtale.....	25
9.2 Samtykkeskjema	28
9.3 Intervjuguide	30
9.4 Nettskjema (Brukerundersøkelse)	33
9.5 Affinitetsdiagram.....	36
9.6 Milepølsplan og Scrum plan	37
9.7 Sprintlogg	38
9.8 Brukertestguide.....	39

1. Introduksjon

Denne rapporten er skrevet som en avslutning på systemutviklingsprosjektet våren 2025 i emnet IN2000 – Software Engineering ved Universitetet i Oslo, og tar for seg hele prosessen knyttet til utviklingen av en applikasjon i et tverrfaglig team. Rapporten inneholder en grundig dokumentasjon av krav og modelleringer, en beskrivelse av det endelige produktet, samt en vurdering av testing, utvikling, arbeidsmetodikk og en refleksjon over de gjennomførte aktivitetene.

Kunstig intelligens (KI) er et rask utviklende felt som påvirker mange aspekt i samfunnet og teknologien i dag – også innen værvarsling. Seks informatikkstudenter; *Jenny Le*, *Laurynas Pilibaitis*, *Daniel Duy Mai*, *Abdirahman Abdulkadir Yusuf*, *Haakon Eide Hilmen* og *Alisa Sorano* har hatt som mål i prosjektet å utforske hvilke muligheter som kan ligge innenfor KI-feltet. Mer spesifikt om KI kan foreslå aktiviteter som passer best for gjeldende værforhold.

I løpet av de siste tre månedene med intensivt samarbeid har teamet utviklet en Android-applikasjon; *OutsAide* (uttales «outside») ved hjelp av smidige metoder, teknikker og verktøy innen programvareutvikling. Applikasjonen retter seg mot unge voksne i alderen 25 – 40 år, som enten ønsker inspirasjon til nye opplevelser, eller som trenger litt ekstra motivasjon for å komme seg ut i naturen og være mer aktive. Ved å koble brukervennlig design med kunstig intelligens og værdata, ønsker vi å bidra til å senke terskelen for å finne passende aktiviteter – ved hjelp av KI.

2. Innledning og prosjektbeskrivelse

2.1 Presentasjon av prosjektet

Våren 2025 etablerte vi en gruppe bestående av seks personer fra forskjellige studieretninger innen informatikk ved Universitetet i Oslo for å samarbeide om utviklingen av en Android applikasjon i emnet IN2000 – Software engineering, ved hjelp av Android Studio med Kotlin og Jetpack Compose. Vi valgte case 5 – «Åpent case bruk av KI», og fokuserte på utviklingen av en applikasjon som benytter kunstig intelligens og API-er fra Metrologisk institutt som løser problemstillingen: *Hvordan kan vi utnytte kunstig intelligens ved å utvikle en applikasjon som forenkler planleggingen av utendørsaktiviteter for personer i aldersgruppen 25-40 år, hvor den inkluderer aktivitetsforslag og værinformasjon på ett sted?* Vi opplevde caset som både nyttig og samfunnsrelevant, samtidig som den ga oss muligheten til å utforske et spennende og aktuelt fagområde – kunstig intelligens! Å utvikle en applikasjon som faktisk kan komme til å praktisk nytte for folk flest, var også en viktig motivasjonsfaktor.

2.2 Kort beskrivelse av løsningen

Løsningen vi har utviklet *OutsAide*, er en applikasjon som foreslår passende utendørsaktiviteter basert på brukerens preferanser og gjeldene værforhold. Brukeren får mulighet til å spesifisere egne ønsker og behov gjennom et enkelt grensesnitt med nedtrekksmenyer, hvor man blant annet kan velge varighet, dato, aktivitetskategori, sted, antall deltakere og søkeradius. Det er også mulighet for å spesifisere varighet, aktivitet eller sted selv. Basert på denne informasjonen, genererer en KI-agent et forslag til en relevant og gjennomførbar aktivitet til det passende været med en utstyrsliste. Brukeren får informasjon om værforholdet den dagen. Applikasjonen tilbyr brukeren å bli omdirigert til Google Maps for å motta veibeskrivelse fra en posisjon bestemt av KI – til stedsforslaget. Det vil også være mulig å lagre søket sitt og legge det til i favorittsiden.

2.3 Presentasjon av prosjektgruppen

Prosjektgruppen vår besto av engasjerte studenter med ulike studieretninger, noe som har gitt oss et tverrfaglig utgangspunkt for prosjektet. Vi utnyttet hverandres styrker og interesseområder for å oppnå en effektiv arbeidsprosess, høy faglig kvalitet og god dynamikk i teamet.

Jenny Le og **Laurynas Pilibaitis** er på andre året på bachelorprogrammet Informatikk: design, bruk og interaksjon. Jenny fungerte som prosjektleder med ansvar for koordinering og organisering av prosjektet, samt utvikling av brukergrensesnitt og UX-design. Laurynas bidro aktivt med UX og UI-design, og fokuserte spesielt på å samle bruker data og skape brukervennlige komponenter.

Daniel Duy Mai og **Alisa Sorano** er på tredje året, mens **Abdirahman Abdulkadir Yusuf** i andre året går bachelorprogrammet Informatikk: programmering og systemarkitektur. Daniel hadde hovedansvar for utviklingen av applikasjonen, med spesielt fokus på backend-løsninger. Abdirahman arbeidet med API-integrasjoner, testing og kvalitetssikring av systemet. Alisa bistod i utviklingen av UI-design og -funksjonalitet, samt i design av maskoten.

Haakon Eide Hilmen er på tredje året sitt i bachelorprogrammet Informatikk: robotikk og intelligente systemer. Han hadde ansvar for dataarkitektur, struktur og teknisk design, og bidro til å sikre at systemet var robust og skalerbart.



Figur 1: Gruppebilde. Fra venstre: Daniel, Abdirahman, Laurynas, Jenny, Alisa og Haakon

3. Brukerdokumentasjon

3.1 Applikasjon og målgruppe

Appen er utviklet med tanke på enkelthet og tilgjengelighet. Brukeren skal med få klikk kunne få forslag til aktiviteter som passer både med været og egne behov. Målgruppen er unge voksne (25 – 40 år), både med og uten barn, som ønsker å finne passende aktiviteter – enten for selv eller med venner og familie. Vi ser for oss at denne målgruppen har midlene for å drive med

utendørsaktiviteter, men har kanskje ikke nok tid eller er kjent med mulighetene som finnes i området. Appen skal gjøre det enkelt for bruker å få forslag som faktisk samsvarer med preferanser og været – uten mye planlegging, men heller med enkle klikk.

3.2 Applikasjonens funksjonalitet og brukerflyt



Figur 2: Skjermbilder av ulike skjermer i OutsAide

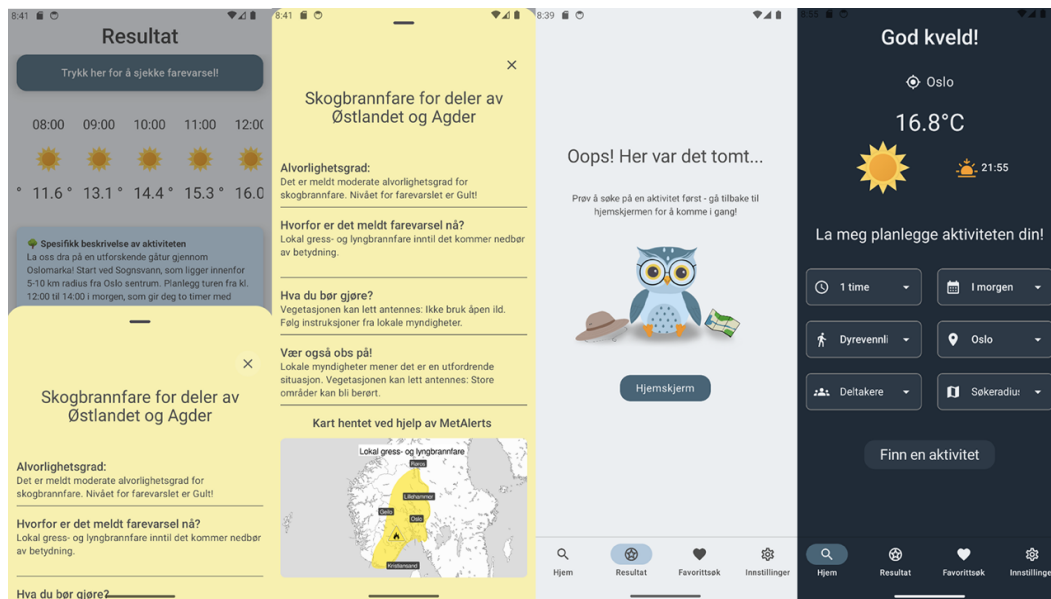
Når applikasjonen åpnes for første gang vil en start-opp skjerm vises med tittelen OutsAide, ugle-maskoten og en «start»-knapp. Etter at bruker har trykket på «start»-knappen vil bruker få opp en bruksanvisning, der de får grunnleggende informasjon for å kunne benytte applikasjonen, og muligheten til å fylle inn navnet sitt. Når de er ferdig, vises hjemskjermen som inkluderer en hilsen til bruker som tilpasser seg med stedsposisjon, et dynamisk ikon av gjelden værforhold, samt nåværende temperatur og tidspunkt for soloppgang/solnedgang. På samme skjerm vises det seks nedtrekksmenyer for å velge preferanser slik at appen kan generere det mest relevante svaret for brukeren. Valgene for nedtrekksmenyer er som følge: varighet, dato, aktivitet, sted, deltakere og søkeradius – hvor kun de tre første menyene er obligatoriske for å kunne få aktivitetsforslag.

Ved klikk av knappen «Finn en aktivitet», vil en lasteskjerm dukke opp med maskoten som sier «Genererer et forslag ...» – hvor KI-agenten bearbeider i bakgrunnen. Deretter vises resultatskjermen som består av informasjon om farevarsler på toppen, temperatur ut dagen, detaljer om aktiviteten, utstyrsliste og farevarsler. Nedover i samme skjerm eksisterer det tre knapper; «Lagre favorittsøk» som lagrer søket i favorittsøk-skjermen, «Nytt svar» som vil generere et nytt forslag basert på samme preferanser, og «Veibeskrivelse» som omdirigerer brukeren til Google Maps for å få veibeskrivelse til stedsforslaget.

Favorittsøk-skjermen, kan bruker enkelt aksessere ved bruk av navigasjonsbaren nederst på skjermen. Skjermen presenterer en oversikt over de lagrede søkene i form av kort. Kortene skal inneholde aktivitet, varighet, sted, og eventuelt, deltakere og søkeradius. Det er også mulig å fjerne kortet fra listen ved å trykke på kryss-knappen. Knappen «Velg dato» omdirigerer bruker tilbake til hjemskjermen med utfylte felter i nedtrekksmenyen. Bruker kan enkelt endre på menyene dersom det er ønskelig, og generere et nytt svar.

Når brukeren prøver å navigere til resultat- eller favorittsøk-skjermen før det har blitt generert eller lagret noe svar, vil en tom skjerm dukke opp som ber bruker om å lage en aktivitet. I midten av skjermen er det en knapp som navigerer til hjemskjermen. Denne skjermen vil også vises dersom det forsøkes å generere et svar uten nettverkstilkobling. Det vil også være en «snackbar» nederst på alle skjermene som sier ifra om manglet nett.

Til slutt tilbyr appen brukerinnstillinger. Her kan bruker lett endre appens fargetema til mørk modus, endre navn, se bruksanvisningen på nytt, og nullstille hele applikasjonen. Vi valgte en fargepalett av dempede og oversiktlige blåfarger som fungerer godt i både lys og mørk modus, noe som kan minne om himmelen.



Figur 3: Skjermbilder av ulike skjerner i OutsAide. De to første skjermene viser informasjon om farevarsler i området. Tredje skjerm bilde viser en tom resultatskjerm. Fjerde skjerm bilde visualiserer applikasjonen i mørk modus.

3.3 Tilgjengelige plattformer

Per dags dato støtter applikasjonen kun Android-plattformen, noe som gjør den kompatibel med Android-smarttelefoner og nettbrett. Grensesnittet er tilpasset ulike skjermstørrelser for å sikre

en optimal brukeropplevelse på tvers av enheter. Applikasjonen er utviklet i Kotlin med Jetpack Compose, som er Googles deklarativ UI-rammeverk spesifikt for Android. Som følge av dette er applikasjonen ikke kompatibel med iOS-enheter, ettersom Jetpack Compose er kun designet for Android.

3.4 Tilgang og bruk

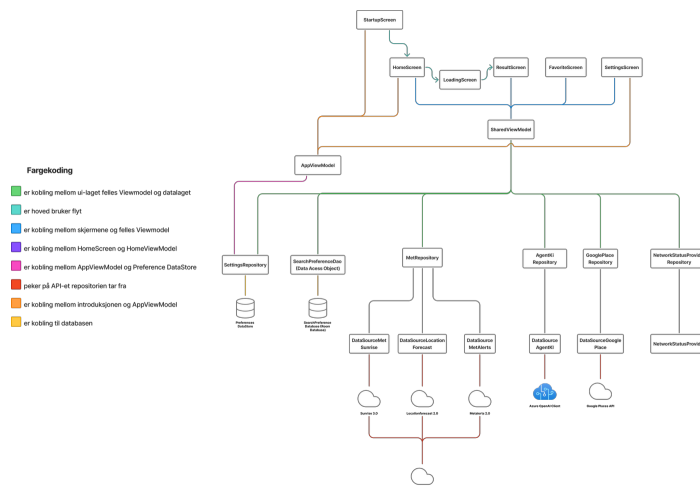
For å aksessere applikasjonen kan bruker laste ned prosjektmappen fra vår Github-side: [IN2000-V25/team-18](https://github.com/IN2000-V25/team-18). Bruker kan deretter åpne prosjektet i Android Studio, og koble den til en enhet som kan være en emulator, eller en fysisk Android-enhet som smarttelefon eller nettbrett. Avslutningsvis kan bruker synkronisere og bygge prosjektet ved å trykke på den grønne «Play»-knappen som vil kjøre applikasjonen.

For å sette opp appen kan SDK konfigureres opp til API 36, men minimumskravet er SDK 26 grunnet Microsoft Azure API-et. Tidligere hadde vi minimumskravet på SDK 24 som førte til at vi fikk opp en feilmelding, og kunne dermed ikke kjøre appen. Vi endret minimumskravet raskt til SDK 26. Vi valgte å sette opp applikasjonen vår mot (targetSDK) API 35 ettersom at vi ønsket å sikre at applikasjonen er tilpasset den nyeste Android-versjonen. Dette minsker teknisk gjeld og drar nytte av ytelsesforbedringer og sikkerhetsoppdateringer som følger med Android 15 (API 35), noe som bidrar til å forbedre brukeropplevelsen av applikasjonen.

4. Produktdokumentasjon

4.1 Applikasjonsstruktur

Applikasjonsstrukturen følger prinsippene i MVVM-arkitektur (Model-View-ViewModel). Alle skjermene «Screen» representerer View-laget som utgjør brukergrensesnittet og henter data fra ViewModels, som fungerer som et mellomledd mellom brukergrensesnittet og datakildene. ViewModel opprettholder UI-tilstanden ved å lagre dataene som presenteres på skjermene, samtidig som den henter informasjon fra «repositories» i Model-laget som kommuniserer med databasene og API-ene.



Figur 4: Skisse av applikasjonsstrukturen

4.2 Kravspesifikasjon

Vi har listet opp funksjonelle og ikke-funksjonelle krav i en tabell for å gi en klar og systematisk oversikt over kravene som applikasjonen må, bør og kan oppfylle. Vi delte inn i ikke-funksjonelle krav i produktkrav, organisatoriske krav og eksterne krav.

Funksjonelle krav	Impl.	
Må	x	Systemet skal kunne gi brukeren forslag til utendørsaktiviteter
	x	Systemet skal vise minst fire nedtrekksmenyer
	x	Systemet skal generere et tekstbasert svar til brukeren
	x	Systemet skal kunne motta og presentere resultater fra KI-agent
	x	Systemet skal kunne lagre favorittaktiviteten
	x	Systemet skal hente værinformasjon basert på valgt dato og sted ved hjelp av vær-API-er
	x	Systemet skal kunne hente værvarsler basert på valgt dato og sted
	x	Systemet skal lagre informasjon om resultatene og brukerinput selv om brukeren navigere til en ny skjerm eller lukker appen
	x	Systemet skal tilby en ny skjerm med resultatet fra KI-agent, vær-advarsler, vær-info, og aktivitet-info skal komme byttes til når resultatet er mottatt
	x	Systemet skal tilby bruker mulighet til å kunne endre på innstillinger til appen
	x	Systemet skal tilby en navigasjonsbar, som skal være hoved metoden til å navigere mellom hjem-skjermen, resultatet og innstillinger
		Systemet skal kunne brukes i 2 språk: Norsk og Engelsk, som skal kunne settes første gang bruker åpner appen Endret til → Systemet skal kun brukes i Norsk
Bør	x	Systemet bør tilby bruker mulighet til å kunne spesifisere sted via søk
	x	Systemet bør tilby brukeren til å lagre favorittsøk som skal kunne brukes senere
	x	Systemet bør tilby bruker evnen til å enkelt slette en, flere eller alle sine lagret favorittsøk
	x	Systemet bør tilby blant valgene sine i nedtrekksmenyer: aktivitet, dato, sted, tid Endret til → Systemet bør tilby blant valgene sine i nedtrekksmenyer: varighet, dato, aktivitet, sted, deltakere og søkeradius
	x	Systemet bør tilby bruker mulighet til å få resultater selv uten at all informasjon er fylt in
	x	Systemet bør gi mulighet for bruker til å sette navn, endre språk, endre mørk modus, og slette all brukerdata i innstillinger

		Endret til → Systemet bør gi mulighet for bruker til å sette navn, endre mørk modus, og slette all brukerdata i innstillinger
Kan		Systemet kan gi mulighet til å gi kartdata
	x	Systemet kan gi bruker forslag til reiser eller turer i formen av hyperlenker til karttjenester (Google-maps)
		Systemet kan presentere bruker med Kart i appen

Ikke-funksjonelle krav	Impl.	
Produktkrav	x	Systemet skal oppfylle WCAG 2.1 til midlende nivå for tilgjengelighet Endret til → Systemet skal oppfylle WCAG 2.1 til minimum eller midlende nivå for tilgjengelighet
	x	Systemet skal gi resultatet fra en “enkel” forespørsel på under 2 sekunder (70% av forespørslene), Systemet skal gi resultat fra en “komplisert” forespørsel på under 10 sekunder (30% av forespørslene) Endret til → Systemet skal gi resultatet på under 11 sekunder
		En ny bruker skal kunne utføre grunnleggende oppgaver i appen uten opplæring innen 5 minutter
	x	Systemet skal være utviklet med minst 10 tester Endret til → Systemet skal være utviklet med minst 10 fungerende tester
	x	Systemet skal fungere på Android API 24 og høyere Endret til → Systemet skal fungere på Android 26 og høyere
	x	Systemet skal støtte skjermstørrelser fra 720 x 1280 og opp til 2560 x 1600 oppløsninger på Android mobiler og nettbrett
	x	Systemet skal bare samle inn og lagre data som er nødvendig for å levere tjenesten, pluss navn
Organisatoriske krav	x	Systemet skal utvikles med Scrumban
	x	Hver Sprint skal ha en ny Scrum-master, og skal vare i 2 uker Endret til → 1 uke senere i prosjektet
	x	Alle aktiviteter laget i sammenheng med systemet skal være ferdig innen 11. mai
	x	Systemet skal utvikles av 6 utviklere, med spesifikke roller og ansvar for systemet
Eksterne krav	x	Oversikt med rapport over alle aktiviteter i sammenheng med systemet skal være fullstendig og innlevert innen 11. mai
	x	Systemet skal være fullstendig og innlevert innen 16. mai

4.3 Kvalitetssegenskaper

For å sikre at applikasjonen var av høy kvalitet, evaluerte vi den ut ifra fire kvalitetsegenskaper: funksjonalitet, brukskvalitet, pålitelighet og universell utforming gjennom ulike metoder. Kjernefunksjonaliteten refererer til applikasjonens evne til å tilby relevante utendørsaktiviteter basert på brukerens preferanse og værddata. Vi evaluerte funksjonaliteten gjennom systematisk gjennomgang, som inkluderte enhetstester og integrasjonstester for å sikre at alle valgte parametere korrekt påvirket aktivitetene som ble foreslått. I tillegg utførte vi geriljatesting, der noen tilfeldige brukere fikk muligheten til å teste funksjonalitetene raskt og uformelt. Deres erfaringer ble dokumentert for å identifisere eventuelle funksjonelle mangler som vi kunne bearbeide videre. Det var spesielt forslaget fra KI-agenten som trakk mye oppmerksomhet da

det var mye informasjon på resultatskjermen. Vi seksjonerte KI-svaret med kategoribaserte overskrifter for å forbedre brukeropplevelsen.

For å evaluere brukskvaliteten gjennomførte vi brukertester med representanter fra målgruppen som vi tidligere hadde intervjuet. Under brukertesten fikk brukerne oppgaven «Lag en plan i morgen med to personer med ønsket aktivitet». Dermed fikk brukerne muligheten til å navigere gjennom applikasjonens grensesnitt, spesifisere sine preferanser, og motta aktivitetsforslag. Deretter ga vi andre oppgaver som «Generer et nytt svar», «Bytt navn», «Nullstill data» og «Lagre aktiviteten». Vi observerte direkte brukernes interaksjoner, og målte tiden det tok for brukerne å utføre oppgavene. Resultatene fra brukertesting viste at applikasjonen var intuitiv og brukervennlig, men enkelte nedtrekksmenyer skapte forvirringer. Menyen «Avstand», som foreslo aktiviteter innenfor en gitt avstand unna stedsposisjonen, ble forvekslet med «distanse», der brukeren eksempelvis trodde de skulle ta en gåtur på 10km. Denne observasjonen fikk oss til å reflektere over menyen «Avstand», og endre navnet på nedtrekksmenyen til «Søkeradius» for å utelukke misforståelser. I tillegg var det spørsmål rundt menyen «Tid», hvor det blant annet var listet: «Om 4 timer», «Om 8 timer», pluss datoer i formatet *M-dd*. Dette gjorde brukeren usikker på om det var dato eller tid som var etterspurt. Vi valgte derfor å endre navnet til «Dato», samt listen i nedtrekksmenyen til «I dag», «I morgen», pluss datoer i formatet *dd.MMM*.

Påliteligheten ble evaluert gjennom feilhåndteringstesting og manuell testprotokoll. I feilhåndteringstesting undersøkte vi eksempelvis hvordan appen kjørte i ulike API-nivåer. Det var ikke mulig å kjøre applikasjonen på API-nivå 25 da vi fikk en feilmelding, men kunne kjøres opptil API-nivå 36. Deretter testet vi manglete input i nedtrekksmenyene for å undersøke om KI-agenten kunne fortsatt generere et spesifikt forslag, noen den ikke kunne. Vi valgte derfor å gjøre nedtrekksmenyene «Varighet», «Aktivitet» og «Dato» til obligatoriske felter. Manuell testprotokoll innebar å lage noen scenarioer som «ingen internett» og «rotering av skjerm». Det viste seg at appen krasjet under internett, men ikke da vi roterte skjermen horisontalt og tilbake til vertikal. Applikasjonen fungerer nå uten internett, men vil ikke kunne genere et forslag, og vil vise en feilmelding på alle skjermer i form av en «snackbar». I tillegg testet vi applikasjonen i ulike skriftstørrelser og oppløsninger, der vi opplevde ingen tap av innhold og informasjon.

Vi ønsket å tilrettelegge en løsning som kan benyttes av flest mulig, og derfor var universell utforming et viktig aspekt i OutsAide. Under vurdering av applikasjonen fokuserte vi på å følge WCAG 2.1-retningslinjene for tilgjengelighet. Vi fokuserte spesielt på kontrastnivåer, og benyttet oss av programmet «Color Contrast Analyser» som anbefalt av Tilsynet for universell utforming av ikt. Dermed tok vi bevisste valg i fargebruk for å kunne nå minimum AA-krav (uutilsynet, 2020). Lesbarhet og intuitiv navigasjon var andre viktige faktorer, som førte til endringer i knappestørrelser og tekstplassering som kan sees i seksjon 5.3 *Designutvikling*. I tillegg har vi brukt de anbefalte tilgjengelighetsverktøyene i Jetpack Compose (Android Developers, 2025), som å ha med beskrivende tekster av alle interaktive elementer som f.eks. knapper, nedtrekksmenyer og bryter, slik at de også kan tolkes av skjermlesere. Dekorative elementer og mellomrom ignoreres for unødvendig opplesning. I koden benyttet vi mye av «contentDescription = null», `semantics{heading()}` og `semantics{hideFromAccessibility()}` (Android Developers, 2025). Dette bidro til at applikasjonen kan benyttes av et bredere spekter av brukere, inkludert personer med syns-, hørsels- og motoriske nedsettelse. I tillegg utviklet vi en bruksanvisning som spilles av ved første kjøring, og som kan gjenspilles i innstillinger.

4.4 API-bruk og utfordringer

Applikasjonen bruker tre API-er fra Meteorologisk institutt for å hente værdata (Meteorologisk Institutt, 2021) (Meteorologisk Institutt, 2024) (Meteorologiske Institutt, 2023), ett fra Azure for å muliggjøre kunstig intelligens i systemet (Microsoft, 2025), og ett fra Google for å få stedsforslag (Google, 2025). Værprognoser ble hentet med *Locationforecast 2.0 API* fra MET. API-et leverer 72-timers værvarsel med intervaller på én time, og deretter på seks timer. Ettersom at API-et var todelt, krevde dette ekstra arbeid for å håndtere og strukturere informasjonen i applikasjonen.

MetAlerts API er et annet API levert av Meteorologisk institutt, og ble benyttet i prosjektet for å hente aktive farevarsler opptil tre dager fram i tid. Hensikten var å gi brukeren oversikt over farevarsler som eventuelt kunne påvirke planlagte aktiviteter. *MetAlerts API* tilbød data i ulike formater, blant annet CAP-format og i en RSS 2.0 - feed, begge basert på XML. For å forenkle integrasjonen valgte vi å benytte en eksperimentell beta-versjon av API-et som leverte i GeoJson-format. Ved bruk av API-et ble to parametere sentral. En av parameterne sørget for å spesifisere fylke, og en annen for å angi det geografiske domenet – enten i land eller marine. Vi bestemte oss for å kun bruke «land»- parameteren, da de fleste aktivitetene er landbaserte.

Likevel kunne marine-området vært en fordel dersom appen skulle tilby aktiviteter på havet som eksempelvis fiske.

Under testing av API oppsto det noen utfordringer. Siden prosjektet i utgangspunktet var begrenset bare i Oslo, var det aldri noe farevarsel for området i testperioden. Dataen ble derfor alltid null og det var vanskelig å forestille seg hvordan et vellykket API-kall faktisk ville sett ut. Selv om API-et ikke krevde obligatoriske parametere, fantes det flere valgfrie parametere for å spesifisere søket ytterligere. For å teste funksjonaliteten hentet vi derfor først varsler for hele Norge. Deretter identifiserte vi et fylke med aktivt farevarsel og gjennomførte et nytt kall med dette fylket som parameter, for å gjenspeile et lignende kall med Oslo som parameter. Postman ble aktivt brukt under testprosessen for å verifisere API-responsene riktig. Til slutt kunne vi implementere hvordan et faktisk farevarsel for Oslo vil se ut.

Azure OpenAI API benyttes for å sende «prompts» (instruksjoner) til språkmodellen GPT-4o som vi selv distribuerte i Azure, noe som muliggjør mottak av tekstbaserte svar fra kunstig intelligens – som er den sentrale funksjonen i applikasjonen (Microsoft, 2025). Vi erfarte imidlertid at det var begrenset dokumentasjon om API-et. Eksempelvis eksisterer det ikke en dokumentasjon for oppsett av KI-agenten i Kotlin, men kun i Java. I tillegg hadde faglærerne ingen kjennskap til Azure da dette er helt nytt element i emnet. Likevel var det enkelt å forstå og implementere i applikasjonen, men vi skulle ønske at det var flere tilgjengelige ressurser for å støtte oss i arbeidet. Andre utfordringer vi opplevde med KI-agent var å få den til å alltid gi riktig svar ut ifra vår preferanse. For å løse dette måtte vi gi den regler for å begrense svaret, og dobbeltsjekke om vi ble fornøyd med resultatet.

Google Places API gjør det mulig å søke og spesifisere stedet brukeren har lyst til å utføre aktiviteten sin under nedtrekksmenyen «Sted» (Google, 2025). API-et gir oss standard informasjon om stedet som navn, adresse, ID i Google Maps, breddegrad (latitude) og lengdegrad (longitude). Brukeren kan skrive «Oslo» og få forslag som «Oslo», «Oslo S» og «Universitetet i Oslo». Etter at brukeren har valgt sted, vil API-et hente detaljene for å identifisere nøyaktig geografisk posisjon. Tidligere brukte vi *Photon OpenStreetMap API* (Photon, u.d.), men vi opplevde denne som ikke pålitelig da serveren kunne krasje ved overbelastning av forespørsler, eller at den var nede. Dermed var *Google Places API* et bedre alternativ som var lett å implementere. Ulempen med Google API-et var vi måtte dekke kostandene dersom det ble gjort over 10 000 kall, noe vi aldri gjorde (Google, 2025). Ettersom

vi fikk koordinater fra *Google Places API*, kunne vi dermed utføre mer presise kall for de andre MET API-ene. For *LocationForecast*, *MetAlerts* og *MetSunrise* ble kallet basert på koordinater, og brukeren fikk dermed lokaliserte resultater for hvilket «sted» de ønsket å utføre aktiviteten.

5. Prosessdokumentasjon

5.1 Utviklingsmetoder og smidige praksiser

I prosjektet valgte vi å implementere Scrumban som smidig metode, en tilnærming som kombinerer elementer fra både Scrum og Kanban (Sommerville, 2021, s. 37). Denne metoden ga oss struktur og fleksibilitet i arbeidsprosessen. Prosjektet ble gjennomført over en periode på 11 uker som bestod av fire faser; planleggingsfasen, utviklingsfasen, testingsfasen og evalueringsfasen. Disse fasene beskrives mer i kapittelet 5.2 *Prosjektets gang*. Arbeidet ble delt inn i seks sprints med en varighet på to uker, unntatt de to siste sprintene, som hadde en varighet på en uke. Dette oppsettet ble bestemt for å gi hvert gruppemedlem muligheten til å fungere som Scrum Master i løpet av en sprint. Hadde prosjektet vart lenger, ville sprintene hatt en varighet på to uker. Hver sprint hadde en dedikert Scrum Master, med klart definerte mål og leveranser. Valget av Scrum Master ble foretatt av metoden «spin the wheel», der vi lot tilfeldigheten bestemme for å sikre rettferdig utvalg og eliminere bias.

Med Scrum, hadde vi en klar sprintstruktur med en Scrum Master, og hver sprint inkluderte daglig-standup, sprint planning, sprint review og sprint retrospektiv. Scrum Master hadde som oppgave å lage en agenda for hvert møte i sprinten, booke møterom, holde oversikt over productbacklogen, og gjennomføre teamaktivitetene. Daglige stand-ups ble gjennomført en gang i uken ettersom møtene våre ble holdt fast på tirsdager og torsdager. Dermed var det mer tid og progresjon som kunne bli tatt opp i møtene. Disse skulle vare maks 6 minutter for å unngå lange oppsummeringer. Hvert gruppemedlem fikk muligheten til å ta opp hva de hadde arbeidet med siden forrige standup, hva de skal jobbe med videre, og hva de trenger hjelp med på ett minutt. Sprint planning ble gjennomført ved hjelp av en Kanban-tavle på GitHub som visualiserte arbeidsflyten. Vi delte opp tavlen i seks deler; Backlog, To do, In progress, Review / Testing, Done this sprint og Past done. Tidligere hadde vi kun «Done», men i senere tid oppstod det forvirringer om hva som ble gjort tidligere eller i denne sprinten, og valgte derfor å dele opp «Done» i to deler. Elementer i «Done this sprint» ble senere flyttet til «Done» i sprint review.

Sprint review ble gjennomført for å reflektere over våre leveranser i sprinten, og diskuterte hva som gikk bra, hva som kan forbedres, og hva som må prioriteres til neste sprint. Igjen brukte vi Kanban-tavlen aktivt for å oppdatere produktbackloggen. I sprint-retrospektiv evaluerte vi arbeidsprosessene våre ved å bruke «Start, Stop, Continue» som er en retrospektiv metode. Vi satte opp disse tre delene i design-verktøyet Figma og la på klistrelapper over hva vi burde starte med å gjøre, hva vi burde stoppe med, og hva vi burde fortsette med. For hver nye retrospektiv gikk vi tilbake til forrige retrospektiv, og vurderte om vi overholdt det som ble avtalt siden sist. Dette ga oss en verdifull innsikt og oversikt over arbeidsprosessen vår. Vi fikk muligheten til å diskutere positive og negative erfaringer i sprinten.

Vi startet sent med å definere utviklingsstrategi i prosjektet. Etter flere merge-konflikter som oppstod under utviklingen av MVP, valgte vi å ta i bruk *Trunk Based Development* (TBD) som ble anbefalt av veilederne da de så vår GitHub nettverksgraf. Vi begynte å sette en standard der alle skulle lage egne gren for hver ny funksjon, og merge denne til *main* etter at den var ferdigutviklet. Vi lagde deretter en ny kanal på Discord som het «jobber-med-main» der vi ga beskjed om noen skulle endre på hovedgrenen, for å unngå at flere jobber med den samtidig. Dette gjorde det enklere å holde koden synkronisert mellom teamet, og reduserte risikoen for at vi jobbet med utdaterte versjoner. Vi fikk bedre oversikt over hva som faktisk ble endret i koden, noe som er nyttig i et tverrfaglig team der flere jobber parallelt med frontend og backend. TBD førte til en smidigere utviklingsprosess, og en mer forutsigbar arbeidsflyt i de siste sprintene.

5.2 Prosjektets gang

Vi startet tidlig med å utvikle en teamavtale for å etablere felles forventinger til prosjektet og sikre godt samarbeid. Deretter utarbeidet vi en milepælsplan for å holde styr på frister, eksamensdatoer, teamaktiviteter og eventuelle viktige arbeidsoppgaver i prosjektet. I tillegg fordelte vi ansvaret for dokumentasjon av sprinter og teamaktiviteter for å sikre god struktur og oversikt.

Vi gjennomførte prosjektet med en planleggingsfase, utviklingsfase, testingsfase, og evalueringsfase gjennom de ulike sprintene. Første sprint var planleggingsfasen, og krevde mye planlegging, diskusjon og forarbeid ettersom målet i sprinten var å få godkjent case. Kravet var

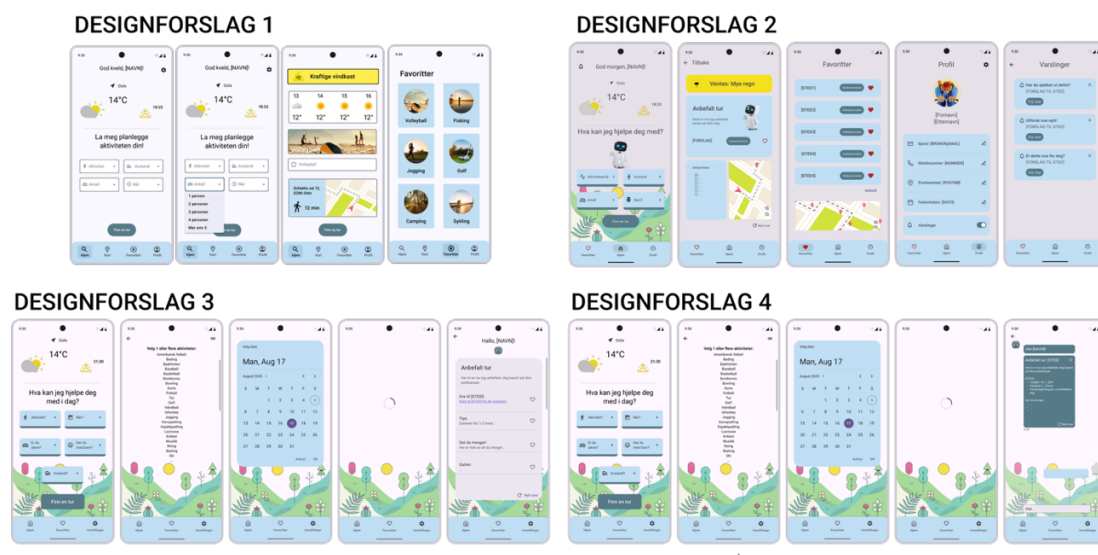
å definere målgruppe, og funksjonelle og ikke-funksjonelle krav. Utviklingen av koden måtte vente, men vi valgte å teste API-ene tidlig for å få en forståelse av hvordan de fungerte, og hvilke data som ble levert. Dette ga oss et bedre grunnlag for å definere de funksjonelle kravene, og informerte beslutninger om arkitekturvalg. Etter at vi hadde fått godkjent case kunne vi starte med brukerundersøkelser og analyser, der vi totalt intervjuet seks brukere gjennom semistrukturerte intervjuer.

Vi befant oss i utviklingsfasen i andre sprint da målet var å ferdigstille minimum viable product (MVP). Vi utviklet raskt fire høyoppløselige wireframes på Figma som vi kunne laste opp i Nettskjema. Vi opprettet et nettskjema som presenterte prototypene for å samle inn tilbakemeldinger fra brukerne, hvor vi totalt fikk 22 svar fra brukere som falt under målgruppen. Deretter kunne vi utvikle applikasjonen, noe vi gjorde sammen gjennom mobbprogrammering i første omgang. Dessverre fikk vi tilgang til KI-agent dagen før demo av MVP, som førte til at hovedfunksjonen ikke var på plass i denne sprinten slik som vi ønsket. Videre i tredje sprint var vi fortsatt i utviklingsfasen for å kunne implementere kunstig intelligens, og bearbeide applikasjonen med tilbakemeldingene vi fikk fra veilederne etter demoen. Hovedutfordringen i sprinten var å få KI til å gi oss en ønskelig output. I tillegg ønsket vi å designe en vennlig maskot som ville representere applikasjonen bedre.

Fjerde sprint var testingsfasen, der vi fokuserte på å teste kjerne funksjonalitetene etter implementering av KI. Vi utførte geriljastest på tre brukere og fikk mange gode tilbakemeldinger vi kunne jobbe videre med i sprinten. Eksempelvis mislikte brukerne at det var dobbelttittel på startskjermen. På den andre siden likte de vær-oppsettet i resultatskjerm, men ønsket at været ble vist timevis istedenfor hver sjetten time. I tillegg måtte vi implementere farevarsel API, og restrukturere koden da vi la merke til at det var mye koderepetisjon. Vi startet med Test-Driven Development og bruk av KI til forbedring, men innså raskt at disse testene ikke kunne ferdigstilles i denne sprinten ettersom det var tidskrevende. Disse aktivitetene ble ferdigstilt i femte sprint som markerte evalueringsfasen i prosjektet. Målet var å samle inn siste tilbakemeldinger om funksjonalitet og brukskvalitet fra brukerne. Det ble gjennomført to brukertester. Oppsummert, var det noen misforståelser angående nedtrekksmenyene som førte til at vi måtte legge til andre menyer og endre på kategoriene i siste liten i sprinten. Endelig i sjette sprint var sprintmålet å ferdigstille applikasjonen og rapportene. Fokuset var å rydde koden, fullføre gjenværende funksjonalitet og skrive ferdig rapportene. Sprinten ble avsluttet med en samlet gjennomgang av innleveringene for å sikre at alle krav var oppfylt.

5.3 Designutvikling

Designprosessen i prosjektet har vært iterativ, hvor vi fokuserte på å utvikle et grensesnitt som var både brukervennlig og funksjonell. Vi startet med å prototype fire enkle designskisser på Figma, hvor formålet var å definere grunnstrukturen i appen som inkluderer navigasjon, funksjoner, skjermer og informasjonsflyt. Designskissene fikk sin inspirasjon fra skissene teamet hadde utviklet under ideutviklingsprosessen Crazy 8 – der alle fikk skissere 8 skjermer per minutt. Deretter opprettet vi et nettskjema som presenterte prototypene for brukerne for å samle inn tilbakemeldinger – og finne ut av hvilket designforslag vi skulle gå videre med. 22 brukere svarte på nettskjemaet, hvor de fleste foretrakk designforslag 1 grunnet sin enkle layout og navigasjon.



Figur 5: Presentasjon av ulike designforslag til målgruppen

Minimum viable product (MVP) ble utviklet i løpet av en to-ukers periode etter brukerundersøkelsen. Applikasjonen fulgte i stor grad samme layout som designforslag 1, men under utviklingen støttet vi på noen designutfordringer. Et sentralt hinder var manglende tilgang til KI-agent, noe som gjorde det vanskelig å designe resultatskjermen, da vi ikke visste hvordan outputen ville se ut. For å komme videre i utviklingen benyttet vi derfor midlertidige data, som vi plasserte i bokser for å simulere sluttresultatet. Videre vurderte vi muligheten for å implementere et søkefelt for å kunne skrive inn spesifikke aktiviteter. Etter flere diskusjoner ble denne funksjonaliteten forkastet, ettersom at vi antok at KI-agent ville ha begrenset evne til å finne eksakte steder som fotballbaner. I stedet valgte vi å tilby forhåndsdefinerte aktivitetsalternativer som «gåtur» og «løpetur», for å simulere KI-en til å generere mer relevante og kreative forslag.

Etter demo av MVP jobbet vi hardt med å videreutvikle appen ettersom vi hadde fått tilgang til KI-agent. Vi utførte geriljatester og fikk mange gode tilbakemeldinger. Blant annet fikk vi tilbakemelding om at oppstartsskjermen var rotete med to overskrifter. Vi valgte å designe en enklere skjerm med en ny maskot for å redusere kognitiv belastning. Tidligere i MVP hadde vi en robot som maskot, men etter en intern diskusjon ble vi enige om at en robot kan virke skremmende, mens en ugle kan virke hyggelig og smart. Uglen skulle representere en «turguide» som ga aktivitetsforslagene til brukeren.

Mot prosjektets avslutning utviklet designet seg i takt med funksjonalitetene etter at de hadde blitt ferdigstilt. Etter hvert som backend og API-integrasjon tok form, måtte flere designvalg tas. Eksempelvis ble flere nedtrekksmenyer utviklet for å kunne gi mer spesifikke svar. Istedenfor spesifikke aktiviteter ble disse endret til kategoriene «Utforskende», «Kulturell», «Mat og drikke», «Dyrevennlig», «Mindfulness» og «Hobbybasert». Tidligere forkastet vi muligheten til å spesifisere aktiviteter gjennom et søkefelt. Etter flere tester med KI, valgte vi å innføre søkefeltet allikevel for ikke å begrense brukerens ønsker og behov. Introduksjonsguiden med GIF-er ble utviklet for å gi brukerne en smidig «onboarding» av applikasjonen. Flere små endringer i fargebruk ble tatt for sikre samsvar med WCAG-kravene.

Første utkast (MVP)



Andre utkast



Endelig applikasjon



Figur 6: Utviklingen av ulike utkast gjennom prosjektet

5.4 Endringer i kravspesifikasjonen

I løpet av prosjektets gang ble det nødvendig å foreta endringer i kravspesifikasjonen etter tilbakemeldinger fra brukere og veiledere, samt tekniske begrensninger og tidsrammer. Dermed

ble enkelte krav justert, forenklet eller fjernet for å sikre gjennomførbarhet. I utgangspunktet skulle applikasjonen støtte både norsk og engelsk, men dette ble endret til kun norsk for å redusere kompleksitet i applikasjonen. Vi hadde som mål å følge WCAG 2.1-retningslinjene fullt ut, men det viste seg at enkelte prinsipper ikke passet eller kunne følges i vår mobilapplikasjon. Vi kan derfor ikke si at applikasjonen vår er universelt utformet, men følger relevante retningslinjer for tilgjengelighet og brukervennlighet. Videre fjernet vi kravet om å ha et innebygd kart og rute siden dette ville kreve mer tid og ressurser. Vi valgte derfor å implementere funksjonen som omdirigerer brukeren til Google Maps for å få veibeskrivelse, da dette var enklere og mer intuitivt.

På den andre siden ble det lagt til flere nye funksjoner underveis basert på brukertesting og teamdiskusjon. Blant annet inkluderte vi visning av soloppgang og solnedgang i hjemskjermen etter inspirasjon av Yr-appen (Metereologiske Institutt, 2023). Vi utviklet to nye nedtrekksmenyer: varighet og sted. «Sted» gir brukeren mulighet til å søke etter aktiviteter i hele Norge. Tidligere begrenset vi oss til kun Oslo, men dette var en funksjonalitet vi klarte å implementere etter brukertesting. Etter flere diskusjoner, kom vi til en enighet at det var mer logisk å lagre søket sitt på favoritt-skjermen enn selve aktiviteten, ettersom forslagene genereres dynamisk av KI-agenten basert på brukerens input og gjeldende værforhold.

5.5 Testing: planlegging, gjennomføring og verktøy

Testing ble planlagt og gjennomført løpende som en integrert del i utviklingsprosessen. Etter at grunnfunksjonene i appen hadde blitt utviklet – som å generere forslag fra KI, lagde vi tester for å sjekke robusthet, avdekke feil og mangler i koden, og optimalisere svar. Koden inkluderer automatiske JUnit tester som kjører hver gang applikasjonen kjøres i Android Studio. Videre testet vi hver funksjon manuelt for å verifisere at applikasjonen oppfylte de funksjonelle og ikke-funksjonelle kravene. Test-Driven Development (TDD) ble benyttet for å teste en funksjon relatert til en *ViewModel*, *Repository* og *API*. Testene ble skrevet først, og deretter utviklet vi en testbar, trygg og stabil kode som fungerte optimalt. Koden i *HomeViewModel* som ble utviklet gjennom TDD ble gjenbrukt i *SharedViewModel*, ettersom vi besluttet oss for å fjerne *HomeViewModel* i slutten av prosjektet. Istedenfor for å opprettholde to separate klasser med identiske funksjoner, valgte vi å kombinere disse til én enkelt klasse (*SharedViewModel*). Logcat i Android Studio ble ofte brukt for feilsøking og logging, mens Postman ble brukt for å teste API-kall. Vi brukte UiO GPT som en del av teamaktiviteten «Bruk av KI til forbedring».

Målet med testingen var å få forslag til forbedringer i koden som nye variabelnavn og refaktoring. Vi opplevde at UiO GPT var god til å forenkle koden, men forverret brukergrensesnittet i *ResultScreen*, mens i *SharedViewModel* brukte vi forslaget om å dele opp koden i mindre deler.

Grunnet av at KI-agenten håndterer logikken til kjernefunksjonaliteten, var det ikke mye funksjonalitet å teste. Likevel var det viktig å skrive tester for å sikre at applikasjonen fungerte pålitelig til enhver tid. Vi mente at det var kritisk å teste API-kall og tilstandene til variablene på tvers av skjermene siden disse var de viktigste funksjonalitetene. Dermed baserte majoriteten av testene seg på disse funksjonene. Eksempelvis testet vi *DataSourceLocationForecast* for å sikre at metodene i *DataSource* fungerte som forventet. Siden *DataSource* sender forespørsler til API-ene, valgte vi å lage tester for hvert eneste API-kall utenom *Azure OpenAI API* og *Google Places API*, på bakgrunn av at vi ikke ønsket å belaste teamet økonomisk. Dersom vi hadde hatt mer tid i prosjektet, hadde det vært ønskelig å lage flere tester for å sikre robusthet. For eksempel kunne vi undersøke hvordan applikasjonen håndterer feilsituasjoner som nettverksfeil.

6. Refleksjon

6.1 Tverrfaglig samarbeid

Et av læringsmålene i emnet IN2000 er å oppnå *kompetanse for å arbeide i team og evne til å reflektere over eget og teamets arbeid i systemutviklingsprosjekter* (UiO, 2025). Å jobbe i tverrfaglig team har vært en verdifull læring for alle involverte. Teamet bestod av medlemmer fra ulike studieretninger, noe som fremmet gjensidig læring ettersom vi kombinerte forskjellige fagområder og kompetanser i utviklingen av applikasjonen. Oppgavene ble fordelt basert på individuelle ferdigheter, men vi la også vekt på å oppmuntre teammedlemmene til å gå utenfor sine komfortsoner og arbeide med oppgaver utenfor sitt primære kompetansefelt. Eksempelvis fikk utviklerne muligheten til å lære designverktøy som Figma og utvikle frontend-løsningen, mens designere fikk anledning til å implementere backend-funksjonalitet. Parprogrammering var spesielt nyttig da den bidro til kunnskapsoverføring. Videre fikk alle anledningen til å prøve seg i rollen som Scrum Master, som er en verdifull erfaring innen prosjektledelse.

Etter å ha gjennomført IN2000 har vi fått erfaring og *kunnskap om følgende aktiviteter innenfor systemutvikling: kravinnsamling og -analyse, design, programmering, testing, samt vedlikehold og videreutvikling* (UiO, 2025). Gjennom det praktiske arbeidet med prosjektet fikk vi erfaring med å anvende smidige utviklingsmetoder som Scrumban. Vi utarbeidet funksjonelle og ikke-funksjonelle krav gjennom grundige analyser, utformet brukervennlige design, implementerte løsninger ved hjelp av Kotlin og Jetpack Compose, samt gjennomførte systematisk testing ved bruk av Test-Driven Development. I tillegg planla vi vedlikehold og videreutviklingen av applikasjonen for å forebygge teknisk gjeld. Vi har forstått hvordan alle disse elementene er tett sammenvevde og påvirker hverandre. Gjennom tverrfaglig samarbeid har vi kombinert ulike perspektiver og kompetanser, som har bidratt med effektiv og kvalitetssikret utviklingsprosess.

6.2 Smidig arbeid og effekt

Et annet læringsmål vi har oppnådd er å *anvende profesjonelle systemutviklingsmetoder, -teknikker og -verktøy* (UiO, 2025). Dette læringsmålet er svært nyttig i arbeidslivet ettersom vi kommer til å oppleve et reelt systemutviklingsprosjekt som faktisk anvender metodene, teknikkene og verktøyene riktig og profesjonelt. Vi organiserte arbeidet med Scrumban som systemutviklingsmetode for å slå to fluer i én smekk. Rettere sagt, ville vi implementere fordelene i Scrum som predefinerte roller og gjennomføre prosjektet sprintvis, mens i Kanban ha en visuell oversikt over arbeidsoppgavene og håndtere endringer effektivt. Scrumban ga oss dermed fleksibilitet i prosjektet. Dersom vi hadde strengt fulgt kun en av metodene, tror vi at det hadde gjort prosjektet noe vanskeligere å håndtere ettersom hver tilnærming har sine egne hovedfokuser og strukturer, noe som faller under læringsmålet *ha kunnskap om de viktigste systemutviklingsmetodene, inkludert deres styrker og svakheter* (UiO, 2025). Eksempelvis følger Scrum en rigid sprintstruktur som er tidskrevende. Det førte til at noen aktiviteter som sprint retrospektiv ble høyere prioritert enn sprint review i prosjektet vårt. Kanban fokuserer på kontinuerlig flyt, og dermed kan det være vanskelig å planlegge langsiktige mål og store leveranser som MVP og fullverdig produkt. I tillegg opplevde vi at Scrumban manglet standardisering da det ikke finnes en «oppskrift». Vi måtte derfor definere mye internt.

Mobbprogrammering og parprogrammering var sentrale arbeidsmetoder i prosjektet. Mobbprogrammering ga oss muligheten til å diskutere koden høyt og lære av hverandre. Gjennom parprogrammering kunne vi samarbeide to og to om konkrete utviklingsoppgaver, som for eksempel ViewModels, noe som skapte mer effektivitet og raskere feildeteksjon.

Verktøy som har vært helt kritiske til å gjennomføre prosjektet har vært Git for versjonskontroll, GitHub for å håndtere Git «repositories» og opprette Kanban-tavle, Figma for planlegging og design, Discord og Teams for kommunikasjon, og Android Studio for kodeutvikling.

Emnet ga oss muligheten til å *få kunnskap om sentrale prosesser og aktører i prosjekt- og teamarbeid som anvender smidige prinsipper* (UiO, 2025). I prosjektet hadde vi kun Scrum Master som rolle da Product Owner (ansvarlig for krav og prioriteringer) ikke var aktuelt her. Likevel hadde vi opprettet et utviklingsteam og designteam for å følge en formell struktur. I praksis hadde denne oppdelingen lite betydning ettersom samarbeidet foregikk på tvers av rollene.

Gjennom arbeidet med prosjektet fikk vi erfaring med ulike smidige prosesser som sprint-planlegging, backlog-håndtering, daglige stand-ups, sprint reviews og sprint-retrospektiv. Sprint-planlegging og sprint-retrospektiv var noe som vi følte hadde en stor effekt i prosjektløpet da vi gjennomførte disse oftere enn de andre aktivitetene. Sprint-planlegging var spesielt viktig for å holde oversikt over sprintmålet, og hvilke oppgaver som skulle prioriteres. Kanban-tavlen gjorde det lettere å fordele arbeidsoppgaver og begrense mengden arbeid i hver fase – bedre kjent som Work in Progress (WIP). Sprint-retrospektiv forbedret teamets arbeid da vi avdekket hva som fungerte og ikke fungerte i sprinten. Det «åpnet» øynene våre ved at vi alltid så en rød tråd i sprintene gjennom «Start, Stop, Continue». Det er definitivt en teknikk vi engasjerer andre å utføre – for læringsutbyttet og refleksjonen er gull verdt her!

6.3 Utviklingsutfordringer og læring

En av de mest tidskrevende utfordringene var knyttet til «local.properties» som er en konfigurasjonsfil. Den inneholder lokale innstillinger til den spesifikke datamaskinen med stien til Android SDK. Selv om vi hadde lagt filen i «.gitignore» (som skal ignorere alle utvalgte filer og mapper), opplevde vi fortsatt problemer, særlig under pull- og merge-operasjoner. Vi antok at utfordringene kunne skyldes av operativsystemene, ettersom teamet bestod av Windows- og Mac-brukere. Etter en grundig undersøkelse, besluttet vi oss å spørre ChatGPT, som foreslo en rekke kommandoer i terminalen i Android Studio. Alle måtte avslutte og åpne Android Studio på nytt, og deretter «pulle» de nyeste endringene og bekrefte at den korrekte «local.properties»-filen dukket opp. Det viste seg å være en vellykket operasjon.

Vi opplevde utfordringer med GitHub, Git og Android Studio. Vi hadde lite kjennskap til grunnleggende Git-arbeidsflyt som inkluderer push, pull, fetch, commit og merge. I retrospektiv skulle vi ha utforsket hvordan Git fungerer i plenum, og blitt enige om en utviklingsstrategi tidlig i prosessen. Det kunne ha spart oss masse tid og krefter. Det var eksempelvis problemer i *main* fordi alle hadde jobbet i den samme filen samtidig, og ikke lagd en ny gren for hver funksjon. Det førte til feil ved «pushing» og «pulling», overskriving av hverandres arbeid, og at man jobbet på utdatert versjon. Det krevde at vi måtte bruke mye tid på å rydde koden og på GitHub. Deretter valgte vi å følge Trunk-Based Development som ga oss bedre kontroll over versjonsloggen, og reduserte risikoen for merge-konflikter betraktelig.

Applikasjonen vår er avhengig av kunstig intelligens for å fungere. Denne KI-agenten var noe som vi fikk dagen før demo av MVP som var relativt sent i prosjektet. Det førte til at mye av arbeidet med koden måtte ventes til denne var klar. Vi ønsket å få KI-agenten implementert før demoen for å kunne teste dens funksjoner og muligheter, noe vi dessverre ikke fikk til.

Videre har vi blitt *kjent med prinsippene for universell utforming* som er et annet læringsmål i emnet (UiO, 2025). I ettertid ser vi at det hadde vært gunstig å gjennomføre testing av fargebruk og kontrastforhold tidligere, enn mot slutten av prosjektet. Dette var relativt enkelt å teste. Vi måtte endre på fargepalletten og fargevalg i komponentene som navigasjonsbar og knapper for å kunne følge retningslinjene for universell utforming.

7. Avslutning

Vi har støtt på mange utfordringer under prosjektet, men vi har likevel tilegnet oss betydelig kunnskap og læring som vi kommer til å ta med oss videre i karrieren vår.

Jenny: «Det er første gang jeg har jobbet i et tverrfaglig team, og med det krever det mye organisering og strukturering tidlig i prosessen. Ikke minst å kjenne hverandres styrker, og svakheter er fornuftig for å kunne utnytte hverandres egenskaper for å forbedre samarbeidet på langsikt, og skape gjensidig læring.»

Laurynas: «Motivasjonen vil definitivt til å svinge gjennom prosjektet for alle. Det er ikke noe man kan kalle på når som helst; den må pleies som en plante. For at motivasjonen skal fortsette

å vokse kreves det kontinuerlig samarbeid og innsats. Dersom gruppen passer på hverandre, så kan planten fortsette med å vokse.»

Daniel: «Det viktigste jeg har lært fra prosjektet er hvor viktig det er å starte tidlig med å avklare hvilken utviklingsstrategi som skal brukes. Det fører til færre kompliserte merge-konflikter, feil avdekkes tidlig, og holder koden oversiktlig og ryddig. Vi startet dessverre sent med å etablere en tydelig strategi som førte til mange komplikasjoner og forvirring.»

Abdirahman: «Jeg lærte å hente og bruke API-er, noe jeg ikke hadde gjort tidligere i praksis. Å arbeide i team med metoden Scrumban har vært en verdifull erfaring som jeg absolutt vil ta med meg videre inn til arbeidslivet. I rollen som Scrum Master utviklet jeg ferdigheter i planlegging, kommunikasjon med teamet, og evnen til å overholde tidsfrister.»

Haakon: «IN2000 har lært meg hvorfor det er kritisk å ha en solid dataarkitektur, inkludert diagrammer og skisser, før man begynner på en oppgave. Mye arbeid kunne blitt forenklet, og mange timer kunne blitt spart dersom man alltid tilnærmet seg en oppgave med et klart overblikk over både hva som skal gjøres og hvordan det skal implementeres.»

Alisa: «Gjennom prosjektet har jeg fått et bedre innblikk i hvordan apputvikling skjer i praksis – fra idé til produkt. Jeg har jobbet litt mer med design-delen, og det har gitt meg mersmak på å jobbe kreativt med visuell utforming. Dette har også styrket lysten til å studere IT-design, noe jeg har vurdert en stund.»

Vi anbefaler at neste års studenter aktivt anvender pensum fra forelesningene og pensumboken gjennom hele prosjektløpet, da det er lett å glemme viktig informasjon underveis. Alt man gjør har en hensikt, og ved å benytte pensum aktivt, blir det lettere å forstå og knytte sammen de ulike elementene, noe som forenkler prosjektgjennomføringen.

8. Kilder og referanser

Android Developers. (2025, Mai 12). *Accessibility in Jetpack Compose*. Hentet fra

<https://developer.android.com/develop/ui/compose/accessibility>

Android Developers. (2025, Mai 13). *Material Design 3 in Compose: Android Developers*. Hentet fra

<https://developer.android.com/develop/ui/compose/designsystems/material3>

Google. (2025, Mars 1). *Places API*. Hentet fra

<https://developers.google.com/maps/documentation/places/web-service>

Meteorologisk Institutt. (2021, Mai 10). *Locationforecast - Documentation*. Hentet fra

<https://api.met.no/weatherapi/locationforecast/2.0/documentation>

Meteorologisk Institutt. (2024, September 13). *Metalerts - Documentation*. Hentet fra

<https://api.met.no/weatherapi/metalerts/2.0/documentation>

Meteorologiske Institutt. (2023, Juni 15). *Sunrise - Documentation*. Hentet fra

<https://api.met.no/weatherapi/sunrise/3.0/documentation>

Microsoft. (2025, Mars 26). *Quickstart: Get started generating text using the legacy completions API*.

Hentet fra <https://learn.microsoft.com/en-us/azure/ai-services/openai/quickstart?tabs=command-line%2Ckeyless%2Ctypescript-keyless%2Centra&pivots=rest-api>

Photon. (u.d.). *Photo - search as you type with OpenStreetMap*. Hentet fra <https://photon.komoot.io/>

Sommerville, I. (2021). *Engineering Software Products - An Introduction to Modern*. Pearson.

Store Norske Leksikon. (2025, April 24). *Smidige utviklingsmetoder*. Hentet fra

https://snl.no/smidige_utviklingsmetoder

UiO. (2025, mai 16). *IN2000 – Software Engineering med prosjektarbeid*. Hentet fra

<https://www.uio.no/studier/emner/matnat/ifi/IN2000/>

Unleash. (2024, Mars 14). *Trunk-based development: process, examples, strategy, implementation*. Hentet

fra <https://www.getunleash.io/blog/how-to-implement-trunk-based-development-a-practical-guide>

utilsynet. (2020, September 10). *Universell utforming av apper*. Hentet fra

<https://www.utilsynet.no/regelverk/universell-utforming-av-apper/230>

9. Vedlegg

9.1 Teamavtale

Teamavtale for team 18 i IN2000

Mål for prosjektet: Utvikle en fungerende applikasjon som bruker KI og API fra MET, og gir bruker forslag basert på brukerinput. Applikasjonen skal være intuitivt og brukervennlig.

Arbeidstid og varighet

Forventet arbeidstimer per uke: 15-26 timer (gjelder uten forelesning)

Startdato og sluttdato: 01. mars 2025 – 14. juni 2025

Oppmøte: Tirsdag 12:15 – 14:00 og torsdag 10:15-12:00

Kommunikasjonskanaler

- Discord
- Teams (chat)

Forventninger

- Det forventes at alle gruppemedlemmer skal bidra til prosjektet, og jobbe jevnt gjennom hele semesteret.
- Det forventes at alle skal ha utført oppgavene sine som har blitt avtalt internt.
- Det forventes at alle skal være tilgjengelig til enhver tid.
- Det forventes at alle skal støtte og hjelpe hverandre, spesielt dersom en henger etter.
- Det forventes at alle skal være med å inkludere hverandre for å skape et godt arbeidsmiljø.
- Det forventes at alle viktige fellesbeskjeder skal meldingen inneholde «@everyone» på Discord. Dermed skal denne meldingen reageres for eksempel med en emoji slik at senderen vet at alle har lest beskjeden.
- Det forventes at det skal bli skrevet et møtereferat til hvert møte.

Konsekvenser av brudd på kontakten

Det vil forekomme konsekvenser dersom en/flere av gruppemedlemmene:

- Kommer sent til møtet uten å si ifra.
- Tar beslutninger på egenhånd uten å ha diskutert med gruppa.
- Ikke har lest seg til relevant pensum som planlagt ut uka.
- Ikke svarer i gruppechatten ila. dagen.
- Ikke holder sine frister.
- Ikke deltar i gruppeprosjektet.

Konsekvensen er å gjøre noe hyggelig for gruppa og ta med noe godt til neste gruppemøte! ☺
Dersom det ofte oppstår brudd på kontrakten skal dette håndteres i henhold til konflikthåndteringen.

Beslutninger

Beslutninger skal tas internt hvor alle gruppemedlemmene er til stede og skal gjøres på en demokratisk måte. Dersom en ønsker anonymisering, skal dette aksepteres uten videre. Anonymisering kan gjøres ved å skrive på en lapp. Det skal ikke tas hensyn til en beslutning dersom en velger og ikke svare, eller ikke er til stede.

Konflikthåndtering

- Viktig at det er åpent for konstruktive tilbakemeldinger.
- Forsøke å komme til bunn i konflikten i løpet av diskusjonen.
- Skal tas 1-1, eventuelt med et annet gruppemedlem.
- Dersom konflikten ikke kan løses internt, skal det tas opp med veileder.

Fravær

Ved planlagt fravær (jobb, legetime etc.) skal det gis beskjed på Discord så fort som mulig. Ved sykdom som skal det også gis beskjed i gruppechatten enten dagen før eller på morgenen.

Andre relevante beslutninger eller unntak

- Et stort gruppeprosjekt kan kreve **lengre arbeidstider** enn forventet antall arbeidstimer per uke, og det innebærer at gruppa sitter på ifi eller andre arbeidssteder etter klokka 16:00 hvis nødvendig. Eventuell helger også hvor møtested blir avtalt.
- Flere andre relevante beslutninger og eller unntak kan komme fortløpende i gruppearbeidet, da er det bare å diskutere dette innad gruppa når den tid kommer.

Samtykke

På bakgrunn av gruppeprosjektet kan det forekomme hendelser som gjerne må dokumenteres gjennom bruk av bilder, videoer og eller lydopptak. Denne samlingen av innsamlet data kan benyttes i både grupperapporten og den individuelle rapporten som skal leveres mot slutten av semesteret. Ved signering av denne arbeidskontrakten, sier du deg enig til å la de andre gruppemedlemmer bruke bilder, videoer og eller lydopptak av deg til videre analyse og rapport ved behov.

Signering av kontrakt

Prosjektnavn

Vi, medlemmene i *IN2000 team 18* bekrefter herved at vi har lest og forstått vilkårene i denne arbeidskontrakten. Vi er enige om å samarbeide og skape et godt arbeidsmiljø innad gruppa i henhold til de fastsatte retningslinjene.

Sted og dato

Gruppemedlem 1 sin signatur

Sted og dato

Gruppemedlem 2 sin signatur

Sted og dato

Gruppemedlem 3 sin signatur

Sted og dato

Gruppemedlem 4 sin signatur

Sted og dato

Gruppemedlem 5 sin signatur

Sted og dato

Gruppemedlem 6 sin signatur

9.2 Samtykkeskjema

Vil du delta i studentprosjektet vårt?

Vi er en studentgruppe i kurset *in2000: Software engineering med prosjektarbeid* ved Institutt

for informatikk, Universitetet i Oslo. Prosjektgruppen består av Jenny Le, Haakon

Eide Hilmen, Abdirahman Abdulkadir Yusuf, Alisa Sorano, Daniel Duy Mai og

Laurynas Pilibaitis. Kursledere er Steffen Almås, epost *1.4.11 Kontrast for ikke-*

tekstlig innhold (Nivå AA) | Tilsynet for universell utforming av ikt. (u.å.). Hentet 16.

mai 2025, fra [https://www.uutilsynet.no/wcag-standarden/1411-kontrast-ikke-tekstlig-](https://www.uutilsynet.no/wcag-standarden/1411-kontrast-ikke-tekstlig-innhold-niva-aa/145)

[innhold-niva-aa/145](https://www.uutilsynet.no/wcag-standarden/1411-kontrast-ikke-tekstlig-innhold-niva-aa/145)

Accessibility in Jetpack Compose. (u.å.). Android Developers. Hentet 16. mai 2025, fra

<https://developer.android.com/develop/ui/compose/accessibility>

, Marthe Nordengen Berntzen, epost marthenb@ifi.uio.no, og Stein Michael Storleer, epost michael@ifi.uio.no.

Vårt prosjekt handler om å utvikle en applikasjon som hjelper brukere med å planlegge utendørsaktiviteter på en rask og enkel måte. Ved hjelp av kunstig intelligens og ulike API-er skal applikasjonen gi relevant værinformasjon, farevarsler og forslag til aktiviteter basert på brukerens preferanser og input.

Personvern

Ingen sensitive personopplysninger (jf. Personvernforordningens artikkel 9 og 10) vil bli innsamlet. Personlige opplysninger om deg vil kun benyttes til formålene beskrevet i dette informasjonsskrivet. Vi behandler opplysningene konfidensielt og i samsvar med personvernregelverket.

Navn og kontaktinformasjon vil holdes skjult og erstattes med fiktive navn, hvis nødvendig. Intervjuet vil kun behandles og transkriberes av oss og kan ettersendes deg ved ønske. Dataen som oppbevares, inkludert anonymisert data, vil ikke bli publisert og vil heller ikke kunne tilbakeføres til deg.

Frivillig deltakelse

Din deltakelse i intervju i prosjektet er frivillig, og du kan trekke deg når som helst. Vi tar opptak og notater gjennom intervjuet. Du kan avslutte intervjuet eller trekke tilbake informasjon som er gitt under intervju eller observasjon når som helst. Da sletter vi all informasjon vi har fått fra deg umiddelbart.

Vi vil slette alle data vi har samlet inn når prosjektet avsluttes 1. juni 2025.

Samtykke

Før intervjuet begynner, ber vi deg om å samtykke til å delta ved å undertegne på at du har lest og forstått informasjonen på dette arket, og ønsker å stille opp til intervju.

Jeg har lest og forstått informasjonen over og gir mitt samtykke til å bli intervjuet

Rettigheter

Vi behandler opplysninger om deg basert på ditt samtykke. Så lenge du kan identifiseres i datamaterialet, har du rett til:

- innsyn i hvilke personopplysninger som er registrert om deg, og å få utlevert en kopi av opplysningene,
- å få rettet personopplysninger om deg,
- å få slettet personopplysninger om deg, og
- å sende klage til Datatilsynet om behandlingen av dine personopplysninger.

Dersom du har andre spørsmål til eller ønsker mer informasjon knyttet til prosjektet – eller ønsker å benytte deg av dine rettigheter, vennligst ta kontakt med kursledelsen i emnet IN2000, Steffen Almås steffa@ifi.uio.no, Marthe Nordengen Berntzen, epost marthenb@ifi.uio.no, og Stein Michael Storleer, epost michael@ifi.uio.no.

Samtykkeerklæring

Før intervjuet begynner, ber vi deg om å samtykke til å delta ved å undertegne på at du har lest og forstått informasjonen på dette arket, og ønsker å stille opp til intervju:

- Jeg, den undertegnede, erklærer at jeg har mottatt og forstått informasjon om studentprosjektet og har fått anledning til å stille spørsmål. Dermed gir jeg mitt samtykke til:
☐ å delta i lydintervju

Sted og dato

Signatur

9.3 Intervjuguide

[INTERVJUGUIDE]

[Introduksjon]

- Presenter kort om deg selv og formålet med intervjuet
 - Formålet er å forstå hvordan brukere planlegger utendørsaktiviteter, hvilke faktorer som påvirker dem, og hvordan en app med KI kan forbedre denne prosessen.
- Innhent samtykke før man tar opptaket av samtalen!
- Estimert tidsbruk 30-45 minutter

[Bakgrunnsinformasjon]

- Kan du fortelle litt om deg selv? (Alder, yrke/utdanning, livssituasjon)
- Hvor ofte planlegger du utendørsaktiviteter?
- Hvilke typer aktiviteter liker du å gjøre? (f.eks. turgåing, sykling, sport, piknik)

[Planlegging av været]

- Hvordan bestemmer du hvilke aktiviteter du skal gjøre?
 - Hva påvirker valget ditt når du bestemmer hva du skal gjøre utendørs? (Vær, tid, anbefalinger, tilgjengelighet osv.)
 - Hvor viktig er været for din beslutning?
 - Hvor viktig er sosial aspektet?
- Er du i noen nettbaserte grupper som driver med utendørsaktiviteter? (F.eks Facebook-grupper eller Discord-kanaler)
- Bruker du i dag noen verktøy eller apper for å hjelpe deg med planlegging? (f.eks. vær-apper, aktivitetsguider)
 - Hva liker/misliker du med disse verktøyene?
- Har du opplevd utfordringer med å planlegge aktiviteter pga. været?

[Passende behov]

- Planlegger du ofte aktiviteter sammen med andre voksne som for eksempel venner eller familiemedlemmer?
- Planlegger du ofte aktiviteter med barn? Hvis ja, hvilke hensyn tar du da?

[Behov og forventinger til en aktivitetsapp]

- Hva kunne en app gjort for å hjelpe deg med å planlegge aktiviteter bedre?
- Hvordan kan appen hjelpe deg hvis du planlegger aktiviteter med barn?
- Hvis du hadde en app som kunne foreslå aktiviteter basert på været, hva ville vært viktig for deg?
 - Hvor viktig er det at forslagene er personlige og tilpasset deg?
- Hva slags informasjon vil du ha i appen? (f.eks. værmelding, farevarsler, transportmuligheter)
- Hvor detaljert skal aktivitetsforslagene være? (f.eks. bare enkle forslag eller med ruteanbefalinger, utstyrsliste, kartoversikt osv.)
- Foretrekker du en app som gir anbefalinger automatisk, eller hvor du kan tilpasse forslagene manuelt?
- Ønsker du at appen skal kunne foreslå aktiviteter basert på hvor mange dere er?

[Brukeropplevelse og funksjonalitet]

- Hvor enkelt bør det være å bruke appen? (f.eks. skal den gi raske anbefalinger med få klikk, eller ønsker du avanserte tilpasningsmuligheter?)
- Hvordan ville du reagert på KI-genererte forslag?
 - Stoler du på automatiserte anbefalinger?
- Ville du likt muligheten til å lagre favorittaktiviteter, eller gå tilbake til dine tidligere søk?
- Hvor viktig er det for deg at appen tilpasser seg dine preferanser over tid?

[Brukerferdigheter]

- Hva slags type mobil bruker du (Android, iPhone)?
- Hvor ofte bruker du mobilen i løpet av dagen?
- Sliter du ofte med å finne det du leter etter i innstillinger?
- Hvordan hadde du vurdert deg selv fra 1-10 på dine mobilbruk ferdigheter?
- Har du brukt KI-apper som ChatGPT, Deepseek eller CoPilot tidligere?

[Avslutning]

- Hvilke utfordringer tror du en slik app kan møte?
- Har du noen tilbakemeldinger eller forslag som kan gjøre denne appen mer nyttig for deg?
- Oppsummering, og takke for deltakelsen!

9.4 Nettskjema (Brukerundersøkelse)

Brukerundersøkelse UX/UI

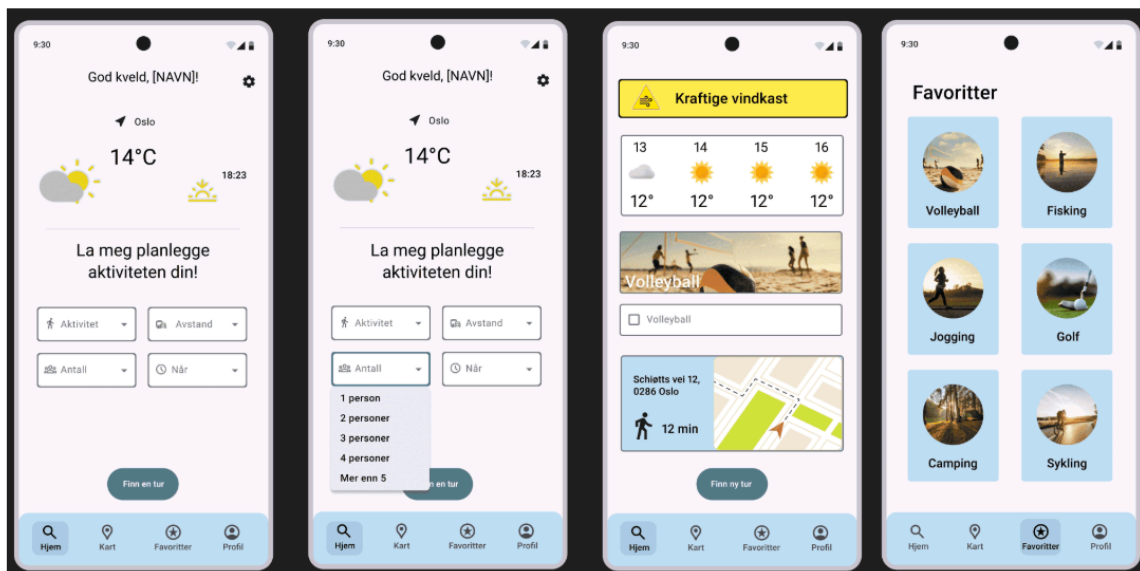
Obligatoriske felter er merket med stjerne *

Navn *

Hei!

Takk for at du er med på spørreundersøkelsen vår! Vi har fått mange gode innspill, og ønsker enda mer! Nå vil vi gjerne høre din mening om våre skisser slik at vi kan forbedre en av de enda mer etter deres behov! Derfor vil det bety masse for oss om du sette av tid og gi oss en tilbakemelding. Det er ønskelig om du kan gi oss en konstruktiv tilbakemelding med setninger istedenfor stikkord.

DESIGNFORLSAG 1



DESIGNFORSLAG 2



DESIGNFORSLAG 3



DESIGNFORSLAG 4



Vennligst ranger følgende designforslag fra mest til minst likt (5 = mest likt, 1 = minst likt)

NB: Du kan kun angi ett designforslag per rangering

	1	2	3	4	5
Designforslag 1 *	☐	☐	☐	☐	☐
Designforslag 2 *	☐	☐	☐	☐	☐
Designforslag 3 *	☐	☐	☐	☐	☐
Designforslag 4 *	☐	☐	☐	☐	☐

Hva liker du best med det designet du rangerte høyest? *

☐ Fargevalg

☐ Layout

☐ Navigasjon

Andre kommentarer:

Er det noe du ville endret på det designet du rangerte høyest?

Hva er det viktigste aspektet ved designet som påvirker din opplevelse? *

☐ Brukervennlighet

☐ Tilgjengelighet

☐ Estetikk

☐ Rask responstid

☐ Enkelhet

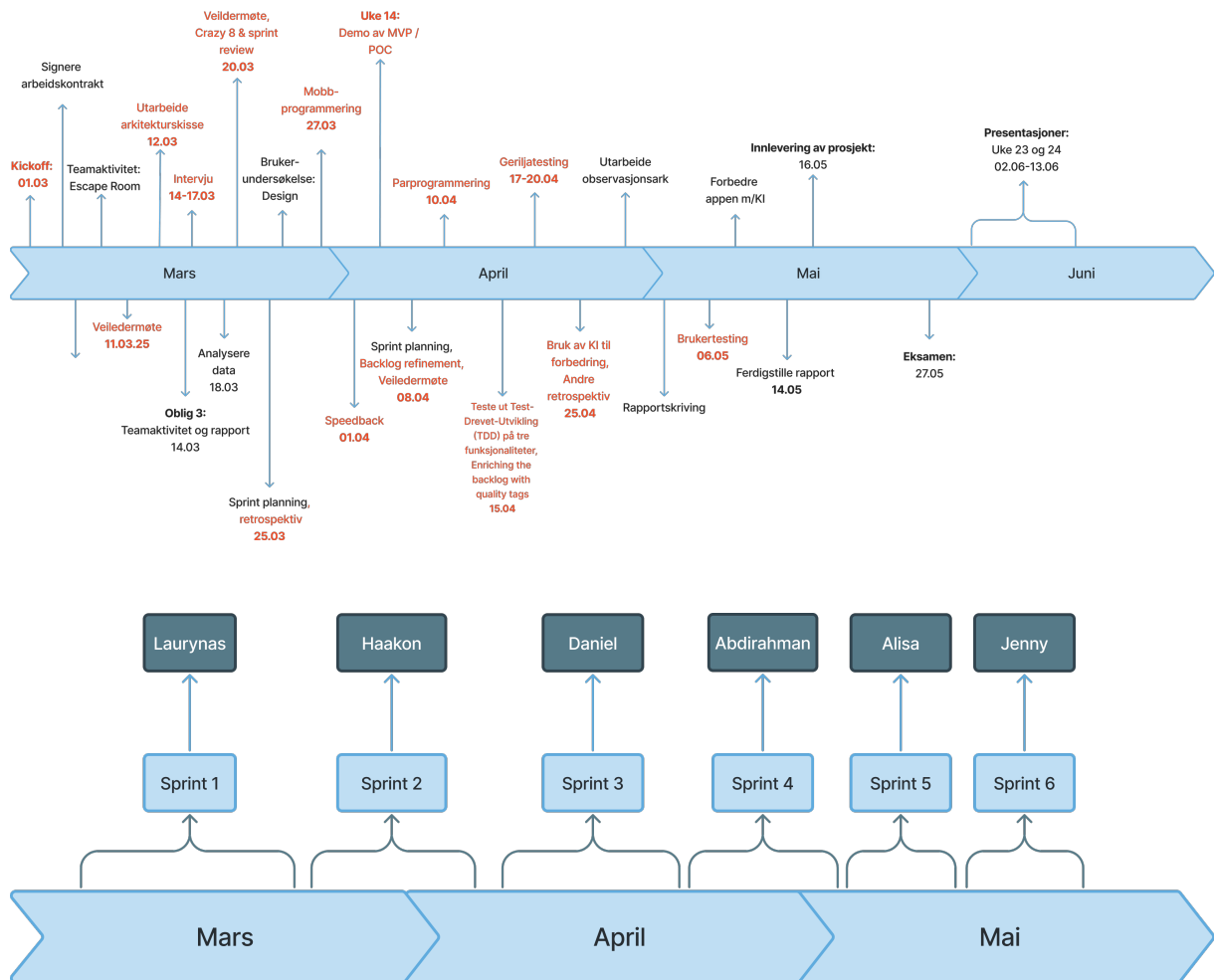
Andre kommentarer:

Har du noe annet du ønsker å legge til?

9.5 Affinitetsdiagram



9.6 Milepælsplan og Scrum plan



9.7 Sprintlogg

Sprint	Start- og sluttdato	Scrum Master	Sprintmål	Viktigste oppgaver	Resultat	Kommentarer / Forbedringer
1	06.03.25 – 23.03.25	Laurynas Pilibaitis	Få godkjent case	1. Send inn søknad om å få case 5 – Åpen case med KI 2. Ferdigstille obligatorisk oppgave 3 3. Utarbeide funksjonelle og ikke-funksjonelle krav 4. Finne målgruppe og holde intervjuer 5. Utarbeide designskisser på Figma 6. Opprette teamavtale og gjennomføre «bli kjent aktivitet»	1. Fikk godkjent case 2. Fikk godkjent obligatorisk oppgave 3 3. «Ferdigstilte» kravspesifikasjonen 4. Utarbeidet dataarkitekturskisse 5. Intervjuet 6 brukere 6. Analyserte intervjuene og lagde flere funksjonelle krav 7. Startet med å utarbeide høyoppløselige wireframes på Figma	Opplevde at mange kom ofte sent til møtet, og måtte derfor endre på teamavtalen og slutte med å «ta med noe godt» som kompensasjon. Samtidig være strengere med å komme til riktig tidspunkt for å holde et mer effektivt møte. Måtte endre på de funksjonelle og ikke-funksjonelle kravene etter obligatorisk oppgave 3 da det var forvirringer rundt kravene.
2	24.03.25 – 06.04.25	Haakon Eide Hilmen	Ferdigstille minimum viable product (MVP)	1. Brukerundersøkelse om designskisser, og prosessering og diskusjon av resultater	1. Brukerundersøkelse ga oss grunnlaget til å utforme appen	Vi fikk tilbakemelding fra veileder at Githuben vår var rotete, og det var fordi under utviklingen av MVP hadde
				2. Ferdigstilling av produkt til MVP 3. Fremvisning av MVP	2. MVP-fremvisning gikk bra, med mye tilbakemelding	vi ikke en struktur, hvor noen hadde jobbet i main mens andre jobbet i egne branches. Vi måtte deretter avtale om hvilken utviklingsmetodikk vi skulle ta i bruk. Vi fikk mange gode tilbakemeldinger fra veilederne etter demo av MVP som fikk oss til å jobbe i riktig retning.
3	07.04.25 – 20.04.25	Daniel Duy Mai	Implementere Agent-KI	1. Implementere og deploye Agent-KI 2. Parprogrammering 3. Design av ny maskot 4. Backlog refinement	1. Klarte å deploye Agent-KI 2. Utviklet ny funksjon «Finn en aktivitet» og får forslag fra KI 3. Ny UI design 4. Ny maskot	Vi skulle ønske at vi utførte teamaktivitetene backlog refinement tidligere ettersom det hadde hjulpet oss med å få en bedre felles forståelse av oppgavene og redusere uklarhet i kravene. Men disse to

						teamaktivitetene kunne kun utføres etter demo av MVP.
4	21.04.25 – 04.05.25	Abdirahman Abdulkadir Yusuf	Testing av applikasjonens hovedfunksjon	1. Utføre geriljatesting 2. TDD 3. Skrive de nødvendige testene 4. Enriching the backlog with quality tags	1. Geriljatesting ble utført, for å raske tilbakemeldinger. 2. Tester ble lagd for å sjekke robushet. 3.	Planen var å utføre teamaktiviteten «Bruk av KI til forbedring» i denne sprinten, men vi innså raskt at det var vanskeligere enn det vi trodde. Denne aktiviteten var tidskrevende og måtte fortsette i neste sprint.
5	05.05.25 – 11.05.25	Alisa Sorano	Samle inn siste feedback gjennom brukertesting	1. Bruk av KI til forbedring 2. Brukertesting med 2 brukere 3. Ferdigstilling av endelig produkt	1. Utført «Bruk av KI til forbedring» og levert nettskjema 2. Refaktorerer 3. Modellere diagrammer 4. Fikse bugs 5. Brukertestet med 2 brukere og fikk mange gode feedback	Skulle opprinnelig ha kodestopp i denne sprinten, men det oppstod mange bugs i siste liten som førte til at vi ikke kunne ferdigstille endelig produkt.
6	12.05.25 – 16.05.25	Jenny Le	Ferdigstille applikasjon og rapport	1. Ferdigstilling av fullverdig produkt 2. Ferdigstilling av rapport 3. Skrive i MODELING.md, README.md og ARCHITECTURE.md 4. Kommentere i koden 5. Leverer rapport og applikasjon	1. Ferdigstilte applikasjon 2. Ferdigstilte prosjektrapport og prosesslogg 3. Ferdigstilte alle markdownfilene	Vi måtte fortsette med å fikse alle små bugs, og utførte små tester for å sikre robusthet og pålitelighet. Det var enormt mye stress i de siste dagene før innleveringen.

9.8 Brukertesting guide

[Brukertesting]

Navn:

Dato:

Funksjonalitet/Oppgaver testet:

- Søking av gåtur, spesifikt i morgen, for 2 personer
- Generere nytt svar (Fra resultatskjerm)
- Bytte navn
- Nullstille all data
- Slett en av favorittaktivitet

Observasjon:

Tilbakemelding/Opplevelse: